UTEC
Universidad Tecnológica

Patrón Factory Method

¿Qué es el Patrón Factory Method?

El **Patrón Factory Method** es un patrón de diseño creacional que define una interfaz para crear objetos en una clase base, pero permite a las subclases cambiar el tipo de objetos que se crearán. En lugar de instanciar objetos directamente con el operador `new`, delegamos la creación de objetos a través de un método fábrica, lo que hace que el código sea más flexible y fácil de mantener.

¿Por qué utilizar el Patrón Factory Method?

Este patrón es útil cuando:

- **Queremos delegar la creación de objetos a subclases**: Esto permite que diferentes subclases generen diferentes tipos de objetos sin modificar el código base.
- **Deseamos encapsular la lógica de creación de objetos**: Mantener la lógica de creación en un único lugar facilita el mantenimiento y la modificación del código.
- **Tenemos varias clases derivadas que comparten una interfaz común**, pero la implementación específica varía entre ellas.

Aplicaciones del Patrón Factory Method

- **Sistemas de manejo de documentos**: Diferentes tipos de documentos (PDF, Word, Excel) pueden ser creados mediante un método fábrica en lugar de instanciar cada tipo de documento manualmente.
- **Interfaces gráficas**: En aplicaciones donde se soportan múltiples estilos de interfaz (por ejemplo, Windows, Mac), podemos utilizar un método fábrica para crear los componentes de la interfaz según la plataforma.
- **Gestión de conexiones de bases de datos**: Permite la creación de diferentes tipos de conexiones de base de datos según la configuración o el entorno.

Ejemplo práctico

Veamos un ejemplo simple para entender cómo funciona el Patrón Factory Method.

Supongamos que estamos creando un sistema para manejar diferentes tipos de vehículos. Tendremos una clase abstracta `Vehiculo` y dos implementaciones concretas: `Auto` y `Moto`. Utilizaremos el Factory Method para crear estos vehículos.

UTEC
Universidad Tecnológica

1. **Interfaz `Vehiculo`**: Definimos una interfaz común para todos los vehículos.

```
public interface Vehiculo {
    void conducir();
}
```

2. **Clases concretas `Auto` y `Moto`**: Implementan la interfaz `Vehiculo`.

```
public class Auto implements Vehiculo {
    @Override
    public void conducir() {
        System.out.println("Conduciendo un auto.");
    }
}

public class Moto implements Vehiculo {
    @Override
    public void conducir() {
        System.out.println("Conduciendo una moto.");
    }
}
```

3. **Clase abstracta `VehiculoFactory`**: Define el método fábrica que será implementado por las subclases.

```
public abstract class VehiculoFactory {
    public abstract Vehiculo crearVehiculo();
}
```

4. **Clases concretas de fábrica `AutoFactory` y `MotoFactory`**: Implementan el método fábrica.

```
public class AutoFactory extends VehiculoFactory {
    @Override
    public Vehiculo crearVehiculo() {
        return new Auto();
```

UTEC
Universidad Tecnológica

```
    }
}

public class MotoFactory extends VehiculoFactory {
    @Override
    public Vehiculo crearVehiculo() {
        return new Moto();
    }
}
```

5. **Uso del Factory Method en la aplicación**:

```
public class Main {
    public static void main(String[] args) {
        VehiculoFactory factory;

        // Crear un auto
        factory = new AutoFactory();
        Vehiculo auto = factory.crearVehiculo();
        auto.conducir();  // Salida: Conduciendo un auto.

        // Crear una moto
        factory = new MotoFactory();
        Vehiculo moto = factory.crearVehiculo();
        moto.conducir();  // Salida: Conduciendo una moto.
    }
}
```

Variaciones y consideraciones

- **Variantes del Patrón Factory Method**: Dependiendo de la complejidad de la creación de objetos, podemos implementar métodos fábrica en la misma clase o en una clase separada.
- **Concurrencia**: Asegúrate de que las instancias creadas a través del Factory Method son seguras en entornos multihilo si es necesario.

UTEC
Universidad Tecnológica

Resumen

El Patrón Factory Method es una herramienta poderosa para manejar la creación de objetos de manera flexible y extensible. Al encapsular la lógica de creación, permite que las aplicaciones sean más fáciles de mantener y escalar.

Más ejemplos de uso

Ejemplo 1: Gestión de conexiones a Bases de Datos

Supongamos que tenemos una aplicación que puede conectarse a diferentes tipos de bases de datos (MySQL, PostgreSQL). El Factory Method nos permite crear la conexión adecuada según el tipo de base de datos.

1. **Interfaz `ConexionBD`**: Define una interfaz común para todas las conexiones.

```
public interface ConexionBD {
    void conectar();
}
```

2. **Clases concretas `ConexionMySQL` y `ConexionPostgreSQL`**: Implementan la interfaz `ConexionBD`.

```
public class ConexionMySQL implements ConexionBD {
    @Override
    public void conectar() {
        System.out.println("Conectando a MySQL...");
    }
}

public class ConexionPostgreSQL implements ConexionBD {
    @Override
    public void conectar() {
        System.out.println("Conectando a PostgreSQL...");
    }
}
```

UTEC
Universidad Tecnológica

3. **Clase abstracta `ConexionFactory`**: Define el método fábrica.

```
public abstract class ConexionFactory {
    public abstract ConexionBD crearConexion();
}
```

4. **Clases concretas de fábrica `MySQLConexionFactory` y `PostgreSQLConexionFactory`**:

```
public class MySQLConexionFactory extends ConexionFactory {
    @Override
    public ConexionBD crearConexion() {
        return new ConexionMySQL();
    }
}

public class PostgreSQLConexionFactory extends ConexionFactory {
    @Override
    public ConexionBD crearConexion() {
        return new ConexionPostgreSQL();
    }
}
```

5. **Uso en la aplicación**:

```
public class Main {
    public static void main(String[] args) {
        ConexionFactory factory;

        // Conectar a MySQL
        factory = new MySQLConexionFactory();
        ConexionBD conexion = factory.crearConexion();
```

UTEC
Universidad Tecnológica

```
        conexion.conectar();  // Salida: Conectando a MySQL...

        // Conectar a PostgreSQL
        factory = new PostgreSQLConexionFactory();
        conexion = factory.crearConexion();
        conexion.conectar();  // Salida: Conectando a PostgreSQL...
    }
}
```

Ejemplo 2: Gestión de notificaciones

Imagina que tienes un sistema que debe enviar diferentes tipos de notificaciones (correo electrónico, SMS). Usar el Factory Method permite crear el tipo correcto de notificación según la preferencia del usuario.

1. **Interfaz `Notificacion`**: Define una interfaz común para todas las notificaciones.

```
public interface Notificacion {
    void enviar(String mensaje);
}
```

2. **Clases concretas `EmailNotificacion` y `SMSNotificacion`**:

```
public class EmailNotificacion implements Notificacion {
    @Override
    public void enviar(String mensaje) {
        System.out.println("Enviando correo electrónico: " + mensaje);
    }
}

public class SMSNotificacion implements Notificacion {
    @Override
    public void enviar(String mensaje) {
        System.out.println("Enviando SMS: " + mensaje);
    }
```

UTEC
Universidad Tecnológica

```
}
```

3. **Clase abstracta `NotificacionFactory`**:

```
public abstract class NotificacionFactory {
    public abstract Notificacion crearNotificacion();
}
```

4. **Clases concretas de fábrica `EmailNotificacionFactory` y `SMSNotificacionFactory`**:

```
public class EmailNotificacionFactory extends NotificacionFactory {
    @Override
    public Notificacion crearNotificacion() {
        return new EmailNotificacion();
    }
}

public class SMSNotificacionFactory extends NotificacionFactory {
    @Override
    public Notificacion crearNotificacion() {
        return new SMSNotificacion();
    }
}
```

5. **Uso en la aplicación**:

```
public class Main {
    public static void main(String[] args) {
        NotificacionFactory factory;

        // Enviar notificación por correo electrónico
        factory = new EmailNotificacionFactory();
        Notificacion notificacion = factory.crearNotificacion();
```

UTEC
Universidad Tecnológica

```
        notificacion.enviar("Hola, este es un correo electrónico.");  // 
Salida: Enviando correo electrónico: Hola, este es un correo 
electrónico.

        // Enviar notificación por SMS
        factory = new SMSNotificacionFactory();
        notificacion = factory.crearNotificacion();
        notificacion.enviar("Hola, este es un SMS.");  // Salida: 
Enviando SMS: Hola, este es un SMS.
    }
}
```

Ejemplo 3: Creación de formatos de archivo

En una aplicación que necesita exportar datos en diferentes formatos (PDF, CSV, XML), el Factory Method permite crear el exportador adecuado según el formato elegido.

1. **Interfaz `Exportador`**: Define una interfaz común para los exportadores.

```
public interface Exportador {
    void exportar(String datos);
}
```

2. **Clases concretas `ExportadorPDF`, `ExportadorCSV`, y `ExportadorXML`**:

```
public class ExportadorPDF implements Exportador {
    @Override
    public void exportar(String datos) {
        System.out.println("Exportando datos como PDF: " + datos);
    }
}

public class ExportadorCSV implements Exportador {
    @Override
    public void exportar(String datos) {
```

UTEC
Universidad Tecnológica

```
        System.out.println("Exportando datos como CSV: " + datos);
    }
}

public class ExportadorXML implements Exportador {
    @Override
    public void exportar(String datos) {
        System.out.println("Exportando datos como XML: " + datos);
    }
}
```

3. **Clase abstracta `ExportadorFactory`**:

```
public abstract class ExportadorFactory {
    public abstract Exportador crearExportador();
}
```

4. **Clases concretas de fábrica `PDFExportadorFactory`, `CSVExportadorFactory`, y `XMLExportadorFactory`**:

```
public class PDFExportadorFactory extends ExportadorFactory {
    @Override
    public Exportador crearExportador() {
        return new ExportadorPDF();
    }
}

public class CSVExportadorFactory extends ExportadorFactory {
    @Override
    public Exportador crearExportador() {
        return new ExportadorCSV();
    }
}

public class XMLExportadorFactory extends ExportadorFactory {
```

UTEC
Universidad Tecnológica

```
    @Override
    public Exportador crearExportador() {
        return new ExportadorXML();
    }
}
```

5. **Uso en la aplicación**:

```
public class Main {
    public static void main(String[] args) {
        ExportadorFactory factory;

        // Exportar como PDF
        factory = new PDFExportadorFactory();
        Exportador exportador = factory.crearExportador();
        exportador.exportar("Datos de ejemplo");  // Salida: Exportando
datos como PDF: Datos de ejemplo

        // Exportar como CSV
        factory = new CSVExportadorFactory();
        exportador = factory.crearExportador();
        exportador.exportar("Datos de ejemplo");  // Salida: Exportando
datos como CSV: Datos de ejemplo

        // Exportar como XML
        factory = new XMLExportadorFactory();
        exportador = factory.crearExportador();
        exportador.exportar("Datos de ejemplo");  // Salida: Exportando
datos como XML: Datos de ejemplo
    }
}
```